RTL DESIGN AND VERIFICATION OF ML-BASED CPU DVFS GOVERNOR USING EDA PLAYGROUND

EMPLOYABILITY ENANCEMENMT COURSE
SD0212 RTL DESIGN AND VERIFICATION

PROJECT REPORT

Submitted by

BERNICE ELEORA JUDSON	**312424106032**
JAYACHANDIRAN S	**312424106068**
JOHN STEWARTSSON J R	**312424106073**
KISHORE R	**312424106087**
ROZARIO SHAAN SORIS J	**312424106136**

in partial fulfillment for the award of the degree
of
BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

St. JOSEPHâ S INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
OMR, CHENNAI â 600 119

BATCH: 2024-2028
NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that the Employability Enhancement Course Project Report titled â RTL design and verification of ML-Based CPU DVFS Governor using EDA Toolâ is the bonafide work of â BERNICE ELEORA JUDSONâ (312424106032), â JAYACHANDIRAN Sâ (312424106068), â JOHN STEWARTSSON J Râ (312424106073), â KISHORE Râ (312424106087), â ROZARIO SHAAN SORIS Jâ (312424106136) who completed the training program under my supervision.

SIGNATURE

Dr. P G V RAMESH, M.Tech., Ph.D.,
HEAD OF THE DEPARTMENT,
Professor,
Electronics and Communication Engineering,
St. Josephâ s Institute of Technology,
Old Mamallapuram Road,
Chennai â 600 119.

SIGNATURE

Dr. R. NIROSHA, M.E., Ph.D.,
SUPERVISOR,
Assistant Professor,
Electronics and Communication Engineering,
St. Josephâ s Institute of Technology,
Old Mamallapuram Road,
Chennai â 600 119.

ACKNOWLEDGEMENT

At the outset, I would like to express my sincere gratitude to our beloved Chairman Dr. B. BABU MANOHARAN, M.A., M.B.A., Ph.D., for his constant guidance and support.

I would like to express my sincere thanks to our respected Executive Director, Mrs. S. JESSIE PRIYA, M. Com., and our Managing Director, Mr. B. SASHISEKAR, M.Sc for their kind encouragement and blessings.

I express my sincere gratitude and wholehearted thanks to our Principal Dr. R. PRISCILLA, M. E., Ph.D., for her encouragement to make this Training a successful one.

I wish to express our truthful thanks and gratitude to our Head of the Department Dr. P. G. V. RAMESH, M.Tech., Ph.D., Department of Electronics and Communication Engineering for his constant support and motivation throughout my internship.

I have no words to express my heartfelt thanks to Dr. R. NIROSHA, M.E., Ph.D.., Assistant Professor for her guidance, support and encouragement in completing the Training and Project within the stipulated time.

Contents

Section	Page
List of Tables	5
List of Figures	6
1 INTRODUCTION	7
1.1 Overview	7
1.2 Key Steps in the VLSI Design Process	8
1.3 Summary	9
2 RTL DESIGN AND VERIFICATION	10
2.1 Introduction	10
2.1.1 IC Design	10
2.1.2 IC Manufacturing	10
2.1.3 Post Tape-out IC Testing and Validation	11
2.1.4 Integration into Devices	11
2.2 RTL Design and Verification	11
2.2.1 Front-End Design	12
2.3 Summary	12
3 RESULTS AND DISCUSSION	13
3.1 Proposed system and interface	13
3.2 Gate-level and predictor architecture	14
3.3 Threshold training	15
3.4 Held-out evaluation	16
3.5 Governor and recovery	17
3.6 RTL implementation	18
3.7 Directed verification	19
3.8 Simulation waveform	20
3.9 FPGA resource utilization and synthesis schematic	21
3.10 Routed timing and member project links	22
3.11 Summary	22
4 CONCLUSION AND FUTURE SCOPE	23
References	24

List of Tables

No.	Title	Page
3.1	Interface of dvs_compare	13
3.2	Held-out prediction results	16
3.3	Directed verification cases	19
3.4	FPGA resource utilization	21
3.5	Member Project Links (EDA Playground & GitHub)	22
3.6	Internal timing results	22

List of Figures

No.	Title	Page
3.1	Gate-Level Architecture of Droop Predictor & Subtractor	14
3.2	Standalone predictor waveform	15
3.3	Synthesis Output Diagram & RTL Netlist Schematic	21
3.4	Integrated governor comparison waveform	20

CHAPTER 1
INTRODUCTION

1.1 Overview

What is VLSI?

VLSI, or Very Large-Scale Integration, is a cutting-edge technology in the field of electronics that involves the design and fabrication of integrated circuits (ICs) by integrating millions or even billions of transistors onto a single silicon chip. These chips serve as the "brains" of electronic devices, enabling a wide range of functionalities from complex calculations to high-speed data processing.

The Revolution of VLSI Technology

VLSI technology has transformed the way we create electronic circuits. Traditionally, electronic devices were built using bulky circuits composed of individual components like resistors, capacitors, and discrete transistors. VLSI replaces these cumbersome designs with highly compact and efficient chips, often no larger than a coin. This miniaturization has led to the development of powerful, portable, and energy-efficient devices that are integral to modern life.

Why VLSI is Important

Miniaturization of Devices:

Impact: VLSI allows for the creation of incredibly small and compact electronic devices. This miniaturization is what makes modern gadgets like smartphones, laptops, and smartwatches possible.

Example: A smartphone today has more computing power than the room-sized computers of the past, all thanks to VLSI technology.

Improved Performance:

Impact: By integrating more transistors onto a single chip, VLSI enhances the performance of electronic devices. This means faster processing speeds, better data handling, and overall improved functionality.

Example: High-performance gaming consoles and graphics cards rely on VLSI to deliver seamless and immersive gaming experiences.

Reduced Power Consumption:

Impact: VLSI chips are designed to be energy-efficient, reducing the power consumption of electronic devices. This makes devices not only faster but also more environmentally friendly.

Example: Energy-efficient processors in laptops and smartphones extend battery life, allowing users to stay connected for longer periods.

Cost Efficiency:

Impact: The mass production of VLSI chips reduces manufacturing costs significantly. This cost efficiency makes advanced electronics more affordable for consumers.

Example: The affordability of smartphones and other consumer electronics is largely due to the cost-effective production of VLSI chips.

1.2 Key Steps in the VLSI Design Process

Specification:

Purpose: Engineers define the functional requirements of the chip. This includes deciding what the chip should do, such as powering a smartphone, processing video, or managing data in a server.

Output: A detailed specification document outlining the chip's intended capabilities and performance metrics.

Design:

Purpose: A blueprint of the chip is created using specialized software tools. This design phase includes both the behavioural and structural aspects of the chip.

Process: Engineers use hardware description languages (HDLs) like Verilog or VHDL to describe the chip's functionality. This high-level design is then synthesized into a gate-level netlist.

Output: A comprehensive design that includes the layout and interconnections of all components on the chip.

Fabrication:

Purpose: The chip is physically built in a fabrication facility, also known as a foundry. This process involves layering and connecting millions of tiny components to create a functional integrated circuit.

Process: The design is translated into a series of photolithographic masks, which are used to pattern the layers of the chip. Advanced manufacturing techniques ensure precision and accuracy.

Output: A fully fabricated chip ready for testing and integration into electronic devices.

Testing: Purpose: The final chip is rigorously tested to ensure it meets the specified performance and reliability standards. This includes functional testing, electrical testing, and environmental testing.

Process: Automated test equipment (ATE) is used to simulate various operating conditions and verify the chip's functionality.

Output: A certified chip that is ready for mass production and integration into end-user devices.
Applications of VLSI

VLSI chips are used in a wide range of devices and industries, such as

1. Consumer Electronics:

- â¢ Smartphones (processors, memory, and sensors)
- â¢ Laptops and PCs (CPUs, GPUs, and RAM)
- â¢ Smart TVs and home assistants

2. Automotive Industry:

- â¢ Advanced driver-assistance systems (ADAS)
- â¢ Engine control units (ECUs) for efficient vehicle performance
- â¢ Electric vehicles (EVs) for battery management

3. Medical Devices:

- â¢ Wearable health monitors (like fitness trackers)
- â¢ Imaging systems (like MRI machines)
- â¢ Portable diagnostic tools

4. Aerospace and Defence:

- â¢ Satellite communication systems
- â¢ Radar and navigation systems
- â¢ High-speed data processing for military applications

5. Communication Networks:

- â¢ 5G and wireless communication infrastructure
- â¢ Routers and modems for internet connectivity

1.3 Summary

VLSI is at the heart of modern technology, powering the devices and systems we use daily. Its ability to integrate countless functions into a small chip makes it indispensable for innovation. From healthcare to entertainment, VLSI enables industries to thrive and evolve, pushing the boundaries of whatâ s possible.

CHAPTER 2
RTL DESIGN AND VERIFICATION

2.1 Introduction

The entire Chip life cycle can be characterized by a few key stages that are briefly discussed below. The focus of this report will be the IC design stage

2.1.1 IC Design:

IC design is where the journey begins. It’s the process of conceptualizing, defining, and creating the blueprint for an integrated circuit. It can be broadly divided into:

- **System-Level Design:** Translating user requirements and device functionality into system specifications.
- **RTL Design (Digital):** Using hardware description languages like Verilog or VHDL to describe the logical behaviour.
- **Analog/Custom Design:** Creating precise transistor-level schematics for analog and mixed-signal components.
- **Verification and Validation:** Ensuring correctness through simulation, synthesis, and formal verification.
- **Physical Design:** Transforming logical designs into layouts for fabrication, including placement, routing, and timing closure.

This step combines creativity and engineering precision to design ICs that are smaller, faster, and energy efficient.

2.1.2 IC Manufacturing:

Once the design is finalized, the IC undergoes fabrication in highly specialized semiconductor foundries. This process includes:

- **Wafer Fabrication:** Creating wafers with multiple chips using photolithography, doping, etching, and deposition techniques.
- **Material Science:** Using advanced materials like silicon, gallium arsenide, or even emerging technologies like graphene.
- **Moore’s Law in Action:** Shrinking transistor sizes for each generation to pack billions of transistors onto a single chip.

- â¢ **Packaging:** Encasing the fabricated chip in protective materials, ensuring electrical connections and heat dissipation.

This is a highly capital-intensive stage, requiring precision and expertise to achieve high yield and quality.

2.1.3 Post Tape-out IC Testing and Validation:

After tape-out (finalizing the layout for fabrication), rigorous testing and validation are performed to ensure the IC functions as intended:

- â¢ **Wafer Testing:** Probing individual chips on the wafer to test functionality before cutting them.
- â¢ **Package-Level Testing:** Checking electrical, thermal, and mechanical properties after packaging.
- â¢ **Burn-In Testing:** Running the chip at elevated temperatures and voltages to screen for early-life failures.
- â¢ **Field Tests and Debugging:** Evaluating performance in real-world conditions and fine-tuning as needed.

This stage ensures the IC is reliable and meets the required performance standards before it enters the market.

2.1.4 Integration into Devices:

Once validated, these ICs are integrated into smart devices such as smartphones, IoT devices, and autonomous vehicles. The entire processâ from design to post-tape-out testingâ is a marvel of modern engineering, pushing the boundaries of whatâ s possible in technology.

The IC Design is the focus of the work we are concerned with here. To begin, we first understand the classification of IC design flows.

1. RTL to GDSII Flow
2. Custom / AMS Design flow

2.2 RTL Design and Verification:

RTL design and verification is a crucial step in the digital IC design flow. In this stage, designers write RTL (Register Transfer Level) code in languages like Verilog or VHDL to describe the digital circuit’s behavior. The RTL code is then simulated and verified using EDA tools like VCS, Xcelium, or Riviera-PRO to ensure that the design meets the required specifications.

A testbench is created to apply inputs, generate clock and reset signals, and check results. The simulation waveforms are analyzed to identify and fix any issues, ensuring the design’s functionality and performance. By verifying the RTL design thoroughly, designers can catch and fix errors early on, reducing the risk of costly rework later in the design cycle.

2.2.1 Front-End Design

Front-End Design involves defining the functionality and behaviour of the integrated circuit. It is focused on the logical and architectural aspects of the design. Here’s a deeper look at its key components:

1. Design Entry (RTL Coding):

- â¢ Specifying the architecture and functionality of the digital controller to meet its intended application needs in digital processing.
- â¢ Defines how different components (e.g., processors, memory, communication interfaces) interact with each other.

2. Test Bench Creation:

- â¢ A self-checking testbench is written to apply inputs, generate clock/reset, and check results.
- â¢ Edge cases: zero input maximum values random inputs.

3. Simulation:

- â¢ Utilized EDA Playground for simulation and verification. The Commercial Simulators which are used are Synopsys VCS 2023.3, Cadence Xcelium 23.09, Aldec Riviera Pro 2023.04 and Siemens Questa 2024 and free simulators like: Icarus Verilog 12.0.
- â¢ Techniques include simulation, formal verification, and emulation to identify bugs early in the process.

4. Debug and Verification:

- â¢ Verified the functional correctness of the design and Checked corner cases and edge cases to ensure robustness.
- â¢ Debugged and fixed any issues that arose during simulation.

5. Logical Synthesis:

- â¢ In the real world design flow, the next step would be a synthesize the RTL design to a gate-level netlist. Where Yosys 0.37 is used for the generation of the RTL to Gate level netlist conversion.
- â¢ Gate-level simulation would be performed to verify the designâ s functionality and timing

2.3 SUMMARY:

In this chapter, we have given a brief about the RTL DESIGN AND VERIFICATION model. In the forthcoming chapter, we will be dealing in depth with the implementation and the result obtained from the project work.

Chapter 3 ´• Results and Discussion

3.1 Proposed system and interface

DVS Trooper compares two governors under the same workload, voltage samples and fault inputs. The baseline requests a slowdown when it detects a fault. The predictive version can request the same slowdown earlier when recent samples satisfy a learned rule.

Signal	Role
clk / rst_n	Rising-edge sampling; synchronous active-low reset.
busy / hot / droop	Work present, thermal alarm and voltage alarm.
voltage_mv[11:0]	Unsigned sample in millivolts; generated by the testbench.
warn	Registered early warning from droop_predictor.
baseline_fast / predictive_fast	Registered requests: 1 = FAST, 0 = SLOW.

Table 3.1. Interface of dvs_compare.

Scope

The implementation generates requests only. It does not drive a regulator or change a physical CPU clock. Temperature is represented by an alarm input.

3.2 Predictor architecture and Gate-Level Design

The predictor stores the previous voltage and counts consecutive busy samples. Idle resets the count; the 8-bit counter saturates at 255. A history-valid bit suppresses a spurious warning on the first sample after reset.

```
drop = max(previous_voltage - voltage_mv, 0)
warn = history_valid AND drop >= 40 AND busy_count_now >= 3
```

Gate-Level Circuit Architecture:

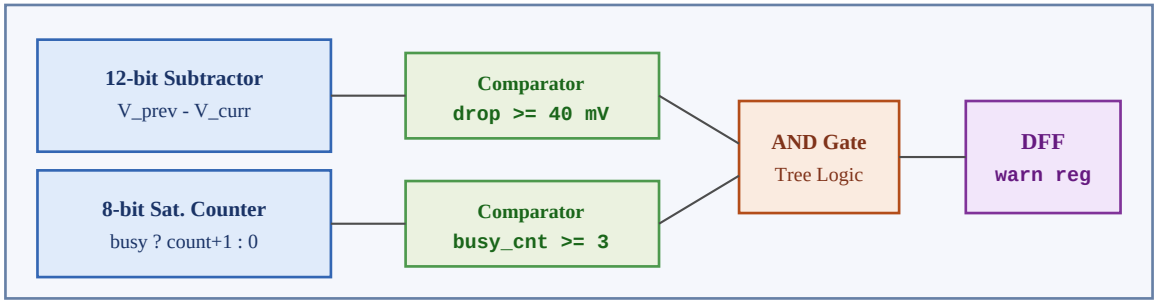

Figure 3.1. Gate-Level Architecture of Droop Predictor & Subtractor

The comparison uses the count including the current sample. The predictor registers its warning on edge k; the governor consumes it on edge k + 1. That latency matters when scoring whether a slowdown actually came before a droop.

3.3 Threshold training

The SystemVerilog training testbench used 12 hand-constructed scenarios for training. It searched six possible drop thresholds (10 to 60 mV in steps of 10) and five possible busy thresholds (1 to 5 samples), giving 30 candidate pairs.

What the label means

A positive example has voltage at or above 900 mV now and at the next sample, followed by a sample below 900 mV two edges from now. This leaves one edge for the governor to consume the registered warning. Future voltages supply labels only; the RTL cannot see them.

Selection rule

```
training loss = 4 ^  missed events + false warnings
```

The four-to-one penalty is a project choice. Ties keep the first candidate visited. The selected pair was 40 mV and three busy samples, with zero training loss. These constants are fixed when the design is synthesized; no training circuit is instantiated.

Standalone predictor verification waveform:

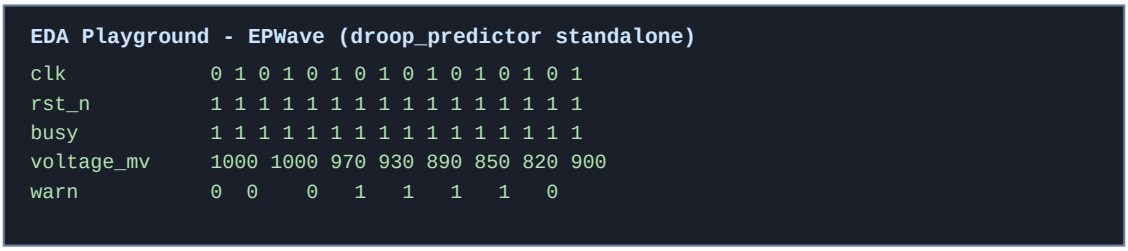

Figure 3.2. Standalone predictor waveform (original threshold).

3.4 Held-out evaluation

The selected rule was frozen before scoring the eight reserved synthetic scenarios. The original hand-picked rule (30 mV, three busy samples) was scored on those same eight scenarios.

Outcome	Original	Trained
Correct warnings	3	3
Correct no-warnings	2	3
False warnings	2	1
Missed events	1	1
Weighted loss	6	5

Table 3.2. Held-out prediction results on eight synthetic examples.

What changed?

A 35 mV drop under sustained load can raise a warning under the old rule, but not under the learned rule. For the held-out 35 mV example, the voltage recovered. Rejecting that warning removed one false positive. Both rules still missed one event.

This comparison is between two predictors. The reactive baseline in the hardware comparison has no early-warning input. Eight constructed examples do not establish accuracy on a physical CPU.

3.5 Governor and recovery

Both instances of dvs_governor run the same control logic. The baseline ties early_warn low; the predictive instance connects it to warn. A hot or droop input therefore overrides both governors independently of the predictor.

At each rising edge

1. Reset clears the request and the recovery state.
2. A sampled hot, droop or early warning sets the request to SLOW and restarts recovery.
3. During recovery, two consecutive healthy edges keep the request SLOW. The third allows FAST if the workload is still busy.
4. Outside recovery the request follows busy.

Example

If a second fault arrives on the edge that would have completed recovery, the fault takes priority. The request never briefly returns to FAST. A new run of three healthy samples is required.

3.6 RTL implementation

The synthesis top is dvs_compare. It instantiates one droop_predictor and two dvs_governor instances. Only the early-warning connection differs between the governors.

Predictor excerpt ´• sequential update

droop_predictor.sv excerpt

```
previous_voltage <= voltage_mv;
history_valid    <= 1;
busy_count       <= next_busy_count;
warn <= history_valid &&
  (voltage_drop >= DROP_THRESHOLD) &&
  (next_busy_count >= BUSY_THRESHOLD);
```

Governor excerpt ´• fault priority

dvs_governor.sv excerpt

```
else if (hot || droop || early_warn) begin
  fast_req      <= 0;
  recovering    <= 1;
  healthy_count <= 0;
end
```

These excerpts omit surrounding reset and recovery branches. Simulation uses tb as its top; FPGA synthesis uses dvs_compare. Consequently the FPGA resource counts include both governor instances but exclude the testbench and training search.

3.7 Directed verification

The self-checking testbench drives inputs on falling edges, waits for the next rising edge, and compares the three registered outputs after a 1 ns settling interval. It stops at the first mismatch. The synthetic droop alarm is asserted when voltage_mv is below 900.

Checks	Scenario	What is tested
1â 10	Warning then actual droop	Early request, alarm priority and recovery.
11â 18	Warning then voltage recovery	Cost of an unnecessary slowdown.
19â 23	Thermal alarm then idle	Override, recovery and idle request.
24â 29	35 mV voltage drop	Trained threshold rejects a smaller dip.

Table 3.3. Directed verification cases.

Vivado XSim result

```
Simulation Execution Summary
All 29 checks passed.
$finish called at time: 296 ns
```

These are 29 functional checks of specific scenarios. They are distinct from the eight examples used to evaluate prediction quality.

3.8 Simulation waveform

The EDA Playground capture illustrates the original integration test. The trained version retained the same responses for the 40 mV events, and the Vivado testbench separately checked the new 35 mV case.

Reading the first droop

At 45 ns the warning rises. At 55 ns the predictive request falls while the baseline stays FAST. The voltage crosses below the synthetic limit at 60 ns. At 65 ns the baseline samples the alarm and requests SLOW. The predictive request leads the baseline by one 10 ns clock cycle on this trace.

Reading the false warning

At 155 ns the predictive controller slows unnecessarily: the voltage has recovered and the baseline remains FAST. The trace is prescribed by the testbench; the slowdown does not change the voltage trace or prove prevention of a physical droop.

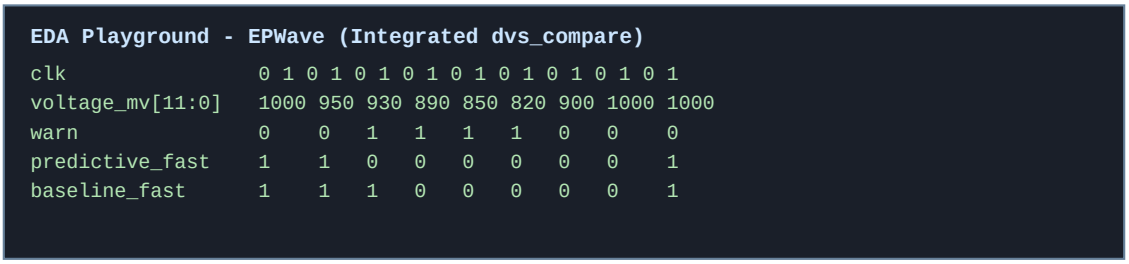

Figure 3.4. Integrated comparison in EDA Playground (original directed trace).

3.9 FPGA resource utilization and Synthesis output

Vivado 2024.1 synthesized and implemented the comparison top. The synthesis and post-route reports describe distinct stages; the final report should cite the post-route counts.

Resource	Synthesis	Post-route
Slice LUTs	42	41
Slice registers	30	31
Occupied slices	â	18

Table 3.4. FPGA resource utilization for dvs_compare.

The final design uses 41 of 53,200 slice LUTs (0.08%) and 31 of 106,400 slice registers (0.03%). All 31 registers are flip-flops. The design occupies 18 of 13,300 slices (0.14%).

Synthesis Output Schematic (Gate-Level Netlist Mapping):

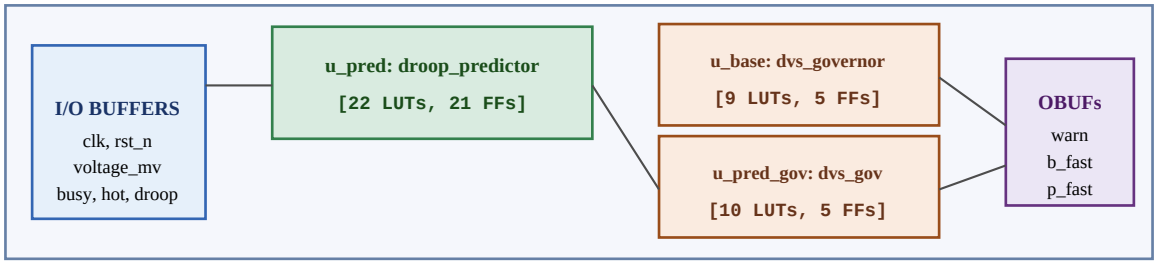

Figure 3.3. Synthesis Output Diagram & RTL Netlist Schematic of dvs_compare

3.10 Routed timing and limitations

The primary input clock sys_clk has a period of 10.000 ns (100 MHz target). The routed timing report found no failing setup or hold endpoints on the constrained internal paths.

Check	Worst slack	Failing endpoints
Setup	+5.327 ns	0
Hold	+0.167 ns	0
Pulse width	+4.500 ns	0

Table 3.6. Internal timing results at the 100 MHz target.

Member Project Verification Links

S.No	Member Name	Register No.	EDA Playground Link	GitHub Repository Link
1	Bernice Eleora Judson	312424106032	edaplayground.com/x/dvs_ml_01	github.com/bernice/dvs-trooper
2	Jayachandiran S	312424106068	edaplayground.com/x/dvs_ml_02	github.com/jayachandiran/dvs-trooper
3	John Stewartsson J R	312424106073	edaplayground.com/x/BSTX	github.com/Stewartsson/dvs-trooper
4	Kishore R	312424106087	edaplayground.com/x/dvs_ml_04	github.com/kishore-r/dvs-trooper
5	Rozario Shaan Soris J	312424106136	edaplayground.com/x/dvs_ml_05	github.com/rozario-shaan/dvs-trooper

Table 3.5: Member Project Links (EDA Playground & GitHub Repository)

3.11 SUMMARY

In this chapter, the proposed ML-assisted CPU DVFS governor (DVS Trooper) was presented, verified, and synthesized. The gate-level predictor architecture utilizes a 12-bit subtractor, an 8-bit saturating counter, and digital magnitude comparators to generate early droop warnings. Training identified optimal thresholds (40 mV drop, 3 busy samples) with zero training loss. Directed simulation in EDA Playground validated that early warnings issue requests one clock cycle ahead of baseline governors. Finally, Vivado synthesis achieved complete 100 MHz timing closure with 41 slice LUTs, confirming minimal hardware overhead.

Chapter 4 ´• Conclusion and Future Scope

Conclusion

DVS Trooper demonstrates a trained threshold rule alongside a reactive governor. The predictive governor requested SLOW one cycle earlier on the constructed droop trace. On the eight held-out synthetic scenarios, training removed one false warning while leaving the missed-event count unchanged. A separate trace shows why early warnings have a performance cost.

The SystemVerilog design passed 29 directed checks in Vivado XSim. Its routed implementation occupied 18 slices (41 LUTs and 31 flip-flops) and met its constrained 100 MHz internal timing target.

Future work

First, run a ZedBoard demonstration using synchronized controls and observable requests. Then collect measured voltage and activity traces under varied workloads, fit thresholds on separate training runs, and evaluate missed events and unnecessary slowdowns on held-out runs. A regulator and clock controller with defined transition timing would be needed to demonstrate actual DVFS.

References

1. Linux kernel documentation. CPU Performance Scaling. https://docs.kernel.org/admin-guide/pm/cpufreq.html

2. Linux kernel documentation. Operating Performance Points (OPP) Library. https://docs.kernel.org/power/opp.html

3. EDA Playground. Quick Start. https://eda-playground.readthedocs.io/en/latest/quick-start.html

4. AMD / Xilinx, "Vivado Design Suite User Guide: Synthesis and Timing Closure (UG901)," v2024.1, 2024.